四分之一的预算被花了两次：有预算上下文打包中的重复证据及一种简单修复

Anonymous ACL submission

摘要

当需要在严格的 token 预算内回答关于长文档的问题时，标准做法是用检索器给文本块排序，给每个块补上相邻文本作为上下文，然后按排名依次拼接，直到预算用完。已有研究主要关注排序这一步；我们关注的是打包这一步，并发现它出人意料地浪费。在 LongBench 上，标准的稠密切块打包把 1,024 token 预算中的 23%、2,048 token 预算中的 29% 花在了上下文里已经出现过的文字上，原因是相邻的高排名块在补全时互相重叠；此外，几乎每个打包片段都在句子中间被截断。我们提出 DedupPack，一种无需训练、可以接在任何排序器之后的打包方法：它把每个选中的片段扩展到句子边界，把相互重叠的扩展合并，使任何文字都不会被付费两次，按下游分词器的精确 token 数并以“回填”方式纳入证据，最后按原文顺序输出。重复率降到 1.4%，省下的预算使答案覆盖率提高 3–4 个百分点。我们在开发集上冻结全部设计选择，然后只在 LongBench 测试集（1,150 个单文档和多文档问答样本）上运行一次，使用三个指令模型（Qwen2.5-7B、Llama-3.1-8B、Qwen2.5-14B）和三档预算（512、1,024、2,048 token）。DedupPack 在全部 9 个模型–预算组合上都优于稠密 BGE 切块打包，三个模型合并后领先 1.9–3.7 F1（每档预算 $p \leq 0.001$），并比结构化的词法配对打包高 2.5–3.1 F1。更强的排序器并不会缩小这一优势：在排序固定的情况下，DedupPack 在 BGE-small、BGE-large 和交叉编码重排序器上分别把切块打包提升 2.8、3.8 和 2.0 F1，并优于 LlamaIndex 的句子窗口检索器、自动合并检索器以及 LongLLMLingua。只用六分之一的输入（2,048 token，完整提示平均 12.5K），DedupPack 在三个模型上保留了完整上下文 99–101% 的 F1（差异均不显著），每个请求的 KV 缓存缩小约 6 倍（如 Qwen2.5-14B 从 2.4 GB 降到 0.4 GB）；在 InfiniteBench 上，1,024 token（仅为 30K 完整上下文的 1/30）保留了 100–104% 的分数。在相同预算下它比 LLMLingua-2 高 15.3 F1。

1 引言

大语言模型（LLM）经常被要求回答关于长文档和文档集合的问题。即使上下文窗口足够长，仍然有理由少给模型一些文本：预填充的计算成本和延迟随提示长度增长，许多部署中的模型窗口较短，而且过长的输入会稀释真正重要的少数段落 (Liu et al., 2024a)。因此在实践中，长上下文常常被压缩到一个严格的预算之内，通常为几百到几千个 token，并以将要阅读它的模型的分词器来计量。

实现这一点的方法分为两类。*提示压缩器* (Li et al., 2023; Jiang et al., 2023, 2024; Pan et al., 2024) 删除被判断为信息量低的 token 或短语。*检索式打包器*把上下文切分成块或片段，用 BM25 或稠密编码器排序 (Robertson and Zaragoza, 2009; Karpukhin et al., 2020; Xiao et al., 2023b)，然后把排名最高的单元依次拼接，每个单元都补上固定窗口的相邻 token，直到预算耗尽。已有研究集中在排序这一步，打包这一步则被当作简单的记账工作。

我们从一个简单的问题出发：*预算到底花在了哪里？*通过测量 LongBench 上标准稠密切块打包所生成的上下文（第 3 节），我们发现相关的块会聚集在一起——能回答问题的那段文字往往包含好几个高排名块——因此它们补全后的窗口互相重叠，同样的句子被输出并付费了好几次。在 1,024 token 预算下，打包文本中有 23% 与同一上下文中的其他文字重复，而且这一比例随预算增大而上升（512、1,024、2,048 token 时分别为 15%、23% 和 29%）。此外，94% 的打包片段在句子中间结束，贪心纳入策略还会留下一部分预算没有用上。在严格的预算下，每一个重复或残缺的 token，都是一个没有用在证据上的 token。

接着我们证明，这种浪费可以在不改动排序器的情况下消除。DedupPack 是一个可以接在任何片段排序器之后的打包器（图 1）。它的核心操作是*通过合并去重*：所有选中片段补全后的区间，只要相接或重叠，就合并成一段，因此原文中的每个 token 至多被付费一次，而一

个新片段的成本就是它实际新增的 token 数。在这一核心之外，DedupPack 按排名顺序、以下游模型的精确 token 数纳入片段，并且回填（在某个片段放不下之后继续尝试后面的片段），把补全对齐到句子边界，最后把合并后的段落按原文顺序逐字输出。这些操作都不需要训练，全部在调用模型之前在 CPU 上完成。

我们在一个尽量不留疑点的协议下评测 DedupPack。所有设计选择（包括排序器）都在一个由 LongBench-E 构建、并去掉了全部 LongBench 测试问题的开发集上确定；之后只在 LongBench 测试集上运行一次。所有方法共享同样的候选片段、提示模板、模型、贪心解码、评分脚本，以及用下游分词器计量的预算。我们测试三个指令模型和三档预算，并报告配对 bootstrap 检验。主要发现如下：

- **标准打包浪费了四分之一的预算。**稠密切块打包的上下文中有 15–30% 是重复内容；DedupPack 把它降到约 1.5%，并在每档预算下把答案覆盖率提高 3–4 个百分点（第 3 节）。
- **消除浪费的效果超过单纯改进排序。**DedupPack 在全部 9 个模型–预算组合上都优于稠密 BGE 切块打包，在 512、1,024、2,048 token 时分别领先 +3.68、+1.92 和 +2.26 F1（三个模型合并，均 $p \le 0.001$），并比结构化的词法配对打包高 2.5–3.1 F1（表 2 和表 3）。
- **更好的排序器并不能消除浪费。**使用 BGE-large 和交叉编码重排序器时，只更换打包器就分别提升 3.8 和 2.0 F1（$p \le 0.001$）；配合重排序器的 DedupPack 是所有压缩配置中最好的，并优于 LlamaIndex 的句子窗口与自动合并检索器、Provence 和 LongLLMLingua（第 6.3 节）。这一趋势在整本书长度的 InfiniteBench 上方向一致（第 6.5 节）。
- **提升与检索器无关。**固定 BGE 排序、只更换打包器，在每档预算下都提升 2.5–2.8 F1；使用纯词法排序器时，DedupPack 仍优于使用相同词法信号的配对打包器（第 6.2 节）。
- **提升不限于问答，而且成本很低。**在 LongBench 全部 11 个带查询的任务上，DedupPack 比切块打包高 5.1 F1（$p < 0.001$）；它在单个 CPU 核上每个样本只需 0.4 秒，比 LongLLMLingua 快约 40 倍（第 6.4 节和第 6.10 节）。
- **六分之一的 token 保住完整上下文的精度。**2,048 token 的 DedupPack 上下文（完整提示平均 12.5K）在三个模型上保留了完整上下文 99–101% 的 F1（配对差分别为 +0.56、−0.35 和 −0.10，均不显著），在 Qwen2.5-32B 上为 103%，在多跳问题上还显著超过完整上下文 3.7 F1（$p = 0.001$）；每个请求的 KV 缓存缩小约 6 倍，在 32B 模型上实测端到端生成时间缩短 6.5 倍；从 7B 到 32B，提升没有随模型变大而缩小（第 6.7 节）；在 InfiniteBench 上，1,024 token 保留了 30K 完整上下文 100–104% 的分数（第 6.5 节）。
- **证据级打包远胜 token 级删除。**在相同的精确预算下，DedupPack 比 LLMLingua-2 高 15.3 F1。
- **去重是真正起作用的成分。**在排序器固定的逐步叠加消融中，仅“合并”一步就拿回了全部提升的 77%（+2.11 F1，$p < 0.001$，在每档预算、每个模型上都显著）；从完整打包器中去掉合并会损失 1.71 F1（$p < 0.001$），其余操作只带来小幅且不显著的提升。重复越多的样本，获益越大（第 6.9 节）。

2 相关工作

提示与上下文压缩。 Selective Context 删除自信息量低的词汇单元 (Li et al., 2023)。LLMLingua 和 LongLLMLingua 用一个小型语言模型估计 token 的重要性，后者还采用了感知问题的由粗到细压缩 (Jiang et al., 2023, 2024)；LLMLingua-2 把保留/删除标签蒸馏到一个双向编码器中 (Pan et al., 2024)。CPC 用训练过的编码器按问题给句子打分 (Liskavets et al., 2025)，RECOMP 和 CompAct 则训练抽取式或生成式压缩器 (Xu et al., 2024; Yoon et al., 2024)。DedupPack 无需训练，从不改动保留段落内部的文字，并且与学习型打分器互补：任何打分器都可以为它提供排序。

检索式上下文构建。 检索增强生成用稀疏 (Robertson and Zaragoza, 2009) 或稠密 (Karpukhin et al., 2020; Xiao et al., 2023b) 检索器对段落排序，并拼接排名靠前的结果 (Lewis et al., 2020; Izacard and Grave, 2021)；考虑多样性的选择方法 (Carbonell and Goldstein, 1998) 和多跳检索 (Xiong et al., 2021; Trivedi et al., 2023) 改进的是选哪些单元。我们研究的是与之互补的问题：在精确预算下，如何把选中的单元变成上下文；并且证明仅这

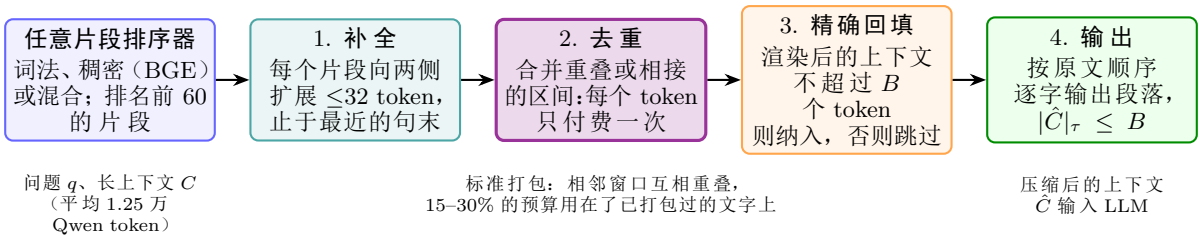


图 1: DedupPack 打包任意排序器返回的片段。每个片段补全到句子边界，重叠的补全区间被合并，使任何文字都不会付费两次；片段按排名顺序、在下游模型的精确 token 预算内以回填方式纳入，结果按原文顺序逐字输出。DedupPack 在预填充之前运行，不修改模型。

B	打包器	重复率↓	句末结束↑	Token 数	答案覆盖↑
512	BGE 切块	14.7	6.7	473	30.4
	配对打包	3.0	6.2	439	30.0
	DedupPack	1.1	70.6	500	33.9
1,024	BGE 切块	22.8	6.3	985	39.0
	配对打包	5.4	6.0	920	36.9
	DedupPack	1.4	68.1	1,014	41.9
2,048	BGE 切块	29.1	6.2	2,010	45.0
	配对打包	7.3	6.1	1,502	41.2
	DedupPack	1.6	62.8	2,032	49.0

表 1: 预算花在了哪里（LongBench 测试集，1,150 个样本，Qwen 分词器；Llama-3.1 在每一列上的差别都在 1 个点以内）。重复率：打包文本中，以该词开头的 8 词序列已在同一上下文前文出现过的词所占比例。句末结束：以句末标点结尾的打包片段所占比例。Token 数：上下文平均长度。答案覆盖：规范化后的标准答案逐字出现在上下文中的样本比例。

一步就能让 F1 变化 2–4 分。已有实践者观察到，重叠切块会返回几乎重复的相邻块，浪费提示预算，一个工业系统因此限制所选块之间的索引间隔 (Sundar and Morabia, 2026)；也有工作测量了 RAG 语料中完全相同的块或文档的去重比例 (Schelpe, 2026)，并有报告指出句子窗口检索不如固定大小切块 (Eibich et al., 2024)。据我们所知，标准打包在单篇长文档内部花在重复文字上的预算比例，以及它对准确率的影响，此前尚未被量化。

长上下文评测与 KV 缓存压缩。 LongBench (Bai et al., 2023) 评测长输入上的问答，而模型对位于长上下文中间的证据利用不足 (Liu et al., 2024a)。KV 缓存方法在生成过程中淘汰、合并或量化缓存的状态 (Xiao et al., 2023a; Zhang et al., 2023; Li et al., 2024; Liu et al., 2024b)。DedupPack 则是在预填充之前缩短输入，因此完整上下文从未被编码。

3 预算花在了哪里？

在任何模型读取之前，我们先测量标准打包器在 LongBench 测试集（第 5 节）上生成的上下文中的三种浪费（表 1）。

重复。 如果某个打包词开头的 8 词序列已经在同一打包上下文的前文出现过，我们就把它计为重复。稠密切块打包在 512、1,024、2,048 token 时分别有 14.7%、22.8% 和 29.1% 的文字是重复的。原因是结构性的：一个问题的高排名候选片段很少均匀分布在文档中，而是聚集在真正相关的段落里，而且相邻片段本身就有重叠（64 token 的片段，步长 32）。再给每个片段两侧各补 32 个 token，相邻窗口就几乎完全重叠。预算越大，纳入的相邻片段越多，重复比例就越大。在选择阶段惩罚冗余的结构化配对打包器能减少重复，但无法消除（3–7%），因为重叠是补全造成的，而不是选择造成的。

残缺句子。 固定窗口打包产生的片段中，只有约 6% 以句末标点结尾；其余 94% 停在句子中间，常常正好停在回答问题的那个短语里。

未用完的预算。 遇到长单元就停止或跳过的贪心纳入策略会留下部分预算没有填满：切块打包平均 1,024 个 token 中剩下 39 个，配对打包剩下 104 个。

后果。 这三种效应叠加起来，降低了答案覆盖率：在每档预算下，标准答案出现在打包上下文中的频率都低于本可以达到的水平。下文介绍的打包器消除了这三种浪费：重复率降到 1.1–1.6%，三分之二的片段在句子边界结束，预算的填充误差在 10 个 token 以内，答案覆盖率比相同预算下的稠密切块打包高 2.9–4.0 个百分点。

4 方法

4.1 问题设定

给定问题 q 和长上下文 C，我们构造满足 $|\hat{C}|_\tau \le B$ 的 $\hat{C}$，其中 $|\cdot|_\tau$ 表示用下游模型的分词器 τ 计算的 token 数。固定的提示模板保持任务指令和问题不变，只把上下文部分替换为 $\hat{C}$。由于预算以下游 token 定义，对每个模型，所有方法都受到完全相同的约束。

4.2 候选片段

上下文被切分成连续的页，每页 256 个正则 token（单词或单个标点），每页再切分成长度为 64 个正则 token、步长为 32 的候选片段；片段不跨页。正则 token 只用于定义候选单元，所有预算计量都使用 τ。我们比较的所有基于片段的方法都使用相同的候选片段。

4.3 排序器

DedupPack 接受任何对片段与 q 打分的函数。我们使用三种，均不接触标准答案。*词法*排序器融合在当前样本所有片段上计算并经 z 标准化的 BM25（k_1=1.2，b=0.75）与 TF-IDF 分数。*稠密*排序器是问题（加 BGE 的查询指令）与片段在 bge-small-en-v1.5 (Xiao et al., 2023b) 下的余弦相似度。*混合*排序器把 z 标准化后的词法分数和稠密分数相加。排名前 60 的片段构成优先列表 P，按分数排序，同分时按位置排序。我们的主配置使用在开发集上选出的混合排序器；第 6.2 节表明提升并不依赖这一选择。

4.4 打包器

通过合并去重（核心）。 高排名片段聚集在相关段落中，它们补全后的区间互相重叠（第 3 节）。DedupPack 把当前选择保存为一组 token 区间，按页和位置排序，并把所有重叠或相接的区间合并成一个。因此重叠的证据只被输出、也只被付费一次；与已选证据相邻的片段，只需支付它新增的 token。

句子对齐的补全。 与标准打包一样，每个片段都会补上上下文，每侧至多 $r = 32$ 个正则 token，且不跨页。不同于固定窗口，DedupPack 在这个半径内遇到最近的句末标点（.、?、!）就停止补全，所以补全后的区间不会比固定窗口长，且通常在句子边界结束。这一操作主要改善可读性；第 6.9 节表明它对准确率的影响很小。

带回填的精确纳入。 片段按优先顺序逐个考虑。如果由当前选择加上该片段渲染出的上下文（补全、合并，并接在前缀“Question: q / Selected evidence:”之后）在 τ 下不超过 B 个 token，就纳入该片段。由于合并可能使新片段几乎不占额外空间，片段的成本取决于已经选了什么；因此纳入判断测量的是渲染并去重后的上下文，而不是把各片段长度相加。如果一个片段放不下，DedupPack 跳过它继续考虑下一个，这样列表后面较短的片段仍然可以用上剩余的预算。

按原文顺序输出。 合并后的段落按原文顺序逐字输出，段落之间用空行分隔。如果重新拼接改变了 token 数，就把上下文截断到 B，从而始终满足 $|\hat{C}|_\tau \le B$。算法 1（附录 A）给出了完整流程。

开发协议。 所有选择——排序器、句子补全还是固定窗口补全、r、列表长度、合并、回填和输出顺序——都在一个 584 个样本的开发集上确定：即 LongBench-E (Bai et al., 2023) 中的 HotpotQA 和 2WikiMultihopQA 部分，并去掉了所有在 LongBench 测试集中出现的问题。开发实验使用 4-bit 的 Qwen2.5-7B 模型，在另一块 GPU 上进行。随后配置被冻结，测试集以文中报告的每个模型和预算只运行一次；没有任何测试结果被用于修改方法。

5 实验设置

数据。 LongBench (Bai et al., 2023) 中全部英文单文档和多文档问答任务：Qasper (Dasigi et al., 2021)（200）、MultiFieldQA-en（150）、HotpotQA (Yang et al., 2018)（200）、2WikiMultihopQA (Ho et al., 2020)（200）、MuSiQue (Trivedi et al., 2022)（200）和 NarrativeQA (Kočiský et al., 2018)（200），共 1,150 个样本。我们报告 LongBench 的 F1 以及六个任务的宏平均；“MH”表示三个多跳任务的平均。

模型与解码。 Qwen2.5-7B-Instruct、Qwen2.5-14B-Instruct (Qwen Team, 2024) 和 Llama-3.1-8B-Instruct (Grattafiori et al., 2024)，以 bfloat16 精度在 vLLM 上运行，把 LongBench 的任务提示放入各模型的对话模板中，使用贪心解码和 LongBench 规定的各任务输出长度。14B 模型读取用 Qwen 分词器打包的上下文，该分词器与 7B 模型相同。

预算。 上下文部分的预算为 $B \in \{512, 1{,}024, 2{,}048\}$ 个下游 token。完整提示平均有 1.25 万个 Qwen token，因此这三档预算大约保留了上下文的 4%、8% 和 16%。

基线。 所有基线都在相同的精确预算和分词器下填充同一个上下文位置。BGE *切块*：在 BGE 下与问题最相似的 64 个片段，每个两侧各补 32 个 token，按相似度顺序在放得下时纳入，并按该顺序输出；这是标准的稠密打包做法。*配对打包*：一种结构化的词法打包器，用词法排序器、词法桥接分数和冗余惩罚选择“查询–支撑”片段对，每个片段用固定的 ±32 窗口补全，依次纳入片段对直到第一个放不下的为止，并按原文顺序输出。它代表“结构化、面向多跳的选择 + 常规打包”。LLMLingua-2 (Pan

et al., 2024)（xlm-roberta-large-meetingbank），要求其把上下文压缩到预算以内；输出放在同样的前缀之后，必要时截断到 B 个下游 token。*完整上下文*：按 LongBench 的做法，把完整上下文从中间截断到 3 万个 token（所有模型使用 32K 窗口），作为参考。第 6.3 节还加入了更强的排序器（BGE-large；在 BGE 前 100 个片段上使用 bge-reranker-v2-m3）以及已发表的系统：LlamaIndex 的句子窗口检索器和自动合并检索器 (Liu, 2022)（官方节点解析器，默认窗口 3、层级 2048/512/128、合并阈值 0.5，用 BGE-small 检索）、用 BGE 排序的不超过 64 词的句子对齐块、应用于 BGE 前 60 个块的 Provence (Chirkova et al., 2025)，以及使用其推荐的问题感知设置的 LongLLMLingua (Jiang et al., 2024)。LongLLMLingua 使用 Llama-3.1-8B-Instruct 作为压缩用的语言模型；它在 1,150 个样本中有 26 个（2.3%）失败，这些样本退回使用上下文开头部分。所有这些方法都在相同的精确预算下打包，各单元按其自身的排名顺序在放得下时纳入。附录 E 给出了早期环境中配对打包器与截断、Selective Context 以及 BM25/TF-IDF 切块打包的比较。

统计方法。 所有比较都按样本配对。我们使用按任务分层的配对 bootstrap (Efron and Tibshirani, 1993)，重采样 10,000 次，随机种子 20260701；统计量是各任务平均差值的宏平均。合并结果在 18 个模型–任务分层内重采样。我们报告 95% 百分位区间和双侧 p 值。

6 结果

6.1 DedupPack 与标准打包的对比

表 2 给出了每个模型和预算下的宏平均 F1，表 3 给出了合并后的配对检验。DedupPack 在全部 9 个模型–预算组合上都优于 BGE 切块打包。三个模型合并后，512 token 时提升 +3.68 F1，1,024 时 +1.92，2,048 时 +2.26，所有区间都不包含 0。按单个模型计，9 个差值中有 8 个在 $p < 0.05$ 下显著（第 9 个是 Qwen2.5-7B 在 1,024 token 时，+1.79，$p = 0.06$；附录 D）。提升在最小预算时最大，因为此时每个被浪费的 token 代价最高。

DedupPack 在每档预算下也比结构化配对打包器高 2.5–3.1 F1（$p < 0.001$），且在全部 9 个组合上都领先。配对打包器是为多跳问题设计的，按片段对选择证据；然而一个逐个片段排序、但打包得当的打包器在多跳任务上同样更好：在 1,024 token 时，DedupPack 在三个模型上的 MH F1 为 46.3、43.9 和 50.3，而配对打包为 42.4、39.6 和 47.0，BGE 切块为 42.7、41.3 和 48.0（附录 C）。

6.2 提升来自打包器，而不是排序器

检验打包器最直接的方法是固定排序。BGE 切块打包和使用 BGE 排序器的 DedupPack 在相同片段上使用相同的 BGE 相似度，唯一的区别是如何把排名靠前的片段变成上下文。更换打包器在三档预算下分别提升 +2.81、+2.48 和 +2.57 F1（表 3），并且按单个模型计在全部 9 个组合上都显著（附录 D）。

反过来，DedupPack 内部选用哪种排序器影响不大。在开发集上选出的混合排序器与 BGE 排序器平均相差不到 0.6 F1，差异不显著（1,024 token 时为 -0.56，$[-1.49, 0.40]$）；两者各在一部分组合中更好。使用纯词法排序器时，DedupPack 在 1,024 token 时比使用相同词法信号的配对打包器高 1.54 F1 [0.68, 2.43]（$p < 0.001$）。总之，提升来自打包操作，并能迁移到词法、稠密和混合排序器上。

6.3 更强的排序器与外部系统

一个自然的问题是：我们发现的浪费是否只对较弱的排序器有影响？表 4 在 Qwen2.5-14B 和 Llama-3.1-8B 上用两种更强的排序器重复了“只换打包器”的实验。使用 BGE-large 时，DedupPack 比切块打包高 3.77 F1 [2.67, 4.88]；使用交叉编码重排序器（在 BGE 前 100 个片段上运行 bge-reranker-v2-m3）时高 2.00 [0.85, 3.12]。两者在每个模型上单独也都显著。因此，更好的排序器并不能消除打包浪费：更好的打包带来的提升，至少与把 BGE-small 换成 BGE-large 或加上重排序器所带来的提升一样大，而且两者可以叠加。配合重排序器的 DedupPack 在两个模型上都是最好的压缩配置，在 Llama-3.1-8B 上甚至超过了 3 万 token 的完整上下文（44.12 对 43.13）。

我们还在相同预算和模板下与已发表的上下文构建系统进行了比较。LlamaIndex 的句子窗口检索 (Liu, 2022) 先检索单个句子，再把每个句子替换为其前后各 3 句的窗口，它比冻结配置的 DedupPack 低 4.18 F1 [2.76, 5.65]；相邻高分句子的窗口严重重叠，这正是第 3 节所说的重复。LlamaIndex 的自动合并检索器在父节点的大部分子块都被检索到时，用父节点替换这些叶子块，它比 DedupPack 低 2.33 [0.94, 3.79]：在固定层级上合并能去掉部分重复，却会纳入大部分内容不相关的整个父节点。LongLLMLingua (Jiang et al., 2024) 低 2.48 [1.02, 3.98]。Provence (Chirkova et al., 2025) 是一个训练过的剪枝器，会从每个检索块中删除无关句子，它与冻结配置接近（+0.17，不显

	Qwen2.5-7B			Llama-3.1-8B			Qwen2.5-14B			
方法	512	1,024	2,048	512	1,024	2,048	512	1,024	2,048	平均
完整上下文（参考）		43.04			43.13			47.59		
LLMLingua-2	–	24.60	–	–	28.43	–	–	27.10	–	–
配对打包	33.88	39.11	40.68	34.05	37.79	39.60	36.73	41.69	44.23	38.64
BGE 切块	33.05	38.85	40.98	33.27	38.99	40.85	36.27	42.34	45.29	38.88
DedupPack（BGE 排序器）	35.64	40.76	43.14	36.16	41.67	43.58	39.21	45.18	48.10	41.49
DedupPack（混合排序器，主配置）	36.74	40.64	43.60	36.44	40.98	42.78	40.44	44.32	47.50	41.49

表 2: LongBench 1,150 个问答样本在 512、1,024 和 2,048 个下游 token 的精确预算下的宏平均 F1。完整上下文参考最多读取 3 万个 token。加粗表示每列中最好的压缩结果。各任务分数见附录 C。

DedupPack 减去	512	1,024	2,048
BGE 切块	+3.68 [2.53, 4.82]	+1.92 [0.87, 2.97]	+2.26 [1.24, 3.29]
配对打包	+2.99 [2.04, 3.94]	+2.45 [1.52, 3.39]	+3.12 [2.18, 4.08]
相同 BGE 排序，不同打包器：			
DedupPack-BGE − BGE 切块	+2.81 [1.88, 3.76]	+2.48 [1.50, 3.45]	+2.57 [1.58, 3.57]

表 3: 三个模型合并后的配对 F1 差值（95% bootstrap 区间）。所有区间都不包含 0（$p \le 0.001$）。各模型单独的检验见附录 D。

排序器	打包方式 / 系统	Qwen-14B	Llama-8B	Δ（合并）
BGE-small	切块	42.34	38.99	
	DedupPack	45.18	41.67	+2.76***
BGE-large	切块	42.55	40.12	
	DedupPack	46.23	43.98	+3.77***
BGE + bge-reranker-v2-m3	切块	44.37	42.21	
	DedupPack	46.45	44.12	+2.00***
LlamaIndex 句子窗口		39.99	36.96	
LlamaIndex 自动合并		41.50	39.13	
句子切块 + BGE		44.24	39.98	
Provence（BGE + 剪枝）		44.27	40.69	
LongLLMLingua		40.66	39.67	
LLMLingua-2		27.10	28.43	
完整上下文（30K）		47.59	43.13	

表 4: 更强的排序器与外部上下文构建系统（LongBench，1,024 token，宏平均 F1）。Δ：在相同排序下 DedupPack 减去切块打包，两个模型合并（***：$p \le 0.001$；每个模型单独也显著）。

著），但明显低于配合重排序器的 DedupPack（+2.80 [1.49, 4.16]）；而 DedupPack 不需要训练剪枝器。最后，用 BGE 打包互不重叠的句子对齐块，从构造上就避免了重复，正如我们的诊断所预测的，它拿回了部分提升；使用相同 BGE 排序器的 DedupPack 仍高出 1.31 F1 [−0.03, 2.67]。

6.4 问答之外的任务

LongBench 还包含另外五个带查询的英文任务：基于查询的会议摘要（QMSum）、三个上下文为一长串示例的少样本任务（TriviaQA、TREC、SAMSum），以及段落检索——判断一段给定摘要描述的是 30 个段落中的哪一个。

	5 个新任务		全部 11 个任务	
方法（1,024 token）	Qwen-14B	Llama-8B	Qwen-14B	Llama-8B
配对打包	58.39	42.94	49.28	40.13
BGE 切块	54.64	39.98	47.93	39.44
DedupPack（BGE）	61.81	47.20	52.74	44.18
DedupPack（混合，冻结配置）	62.56	49.67	52.61	44.93
完整上下文（30K）	63.10	46.59	54.64	44.70

表 5: LongBench 中全部带查询的英文任务：六个问答任务，加上基于查询的摘要（QMSum）、少样本学习（TriviaQA、TREC、SAMSum）和段落检索（PassageRetrieval-en），每个任务 200 个样本。各任务分数见附录 C。

我们用冻结配置原样运行这些任务（表 5）。在全部 11 个带查询的任务上，两个模型合并后，DedupPack 比切块打包高 5.08 F1 [4.18, 6.00]，比配对打包器高 4.06 [3.21, 4.89]（均 $p < 0.001$，每个模型单独也显著）；在 Llama-3.1-8B 上它高于 3 万 token 的完整上下文（44.93 对 44.70）。新任务上的提升主要来自段落检索（Qwen2.5-14B 上准确率 95.6 对 56.6，Llama-3.1-8B 上 59.5 对 12.5）：在这个任务中，重复的补全把候选段落挤出了上下文，正确段落常常因此缺失。在摘要和少样本任务上，只需要一些相关材料而不是某个特定段落，DedupPack 与切块打包持平（+0.25，不显著）：在几乎没有东西可以找回的任务上，去重也不会造成损害。

6.5 第二个基准：整本书长度的上下文

为了检验这些发现能否推广到 LongBench 之外，我们在 InfiniteBench 的英文小说问答和选择题任务 (Zhang et al., 2024) 上运行冻结配置，其上下文长度约为 LongBench 的十倍（表 6）。由于相关段落分散在整部小说中，切块打包在这里只有 8–11% 的上下文是重复的（LongBench 上为 23–29%），因此诊断预测提升会更小。我们没有修改任何参数。冻结配置的 DedupPack 在四个模型–预算组合中的三个上是最好的方法，合并后比配对打包器高 4.54 F1 [3.02, 6.02]（$p < 0.001$）。它相对稠

	Qwen2.5-14B		Llama-3.1-8B	
方法	1,024	2,048	1,024	2,048
配对打包	33.71	34.61	32.80	36.55
BGE 切块	36.58	37.44	36.81	40.15
句子切块 + BGE	36.36	37.69	36.09	39.99
DedupPack（混合，冻结配置）	38.03	38.45	38.36	40.98
DedupPack（BGE）	36.76	37.63	36.56	41.49
完整上下文（30K）	36.44		38.16	

表 6: InfiniteBench 的 En.QA 和 En.MC (Zhang et al., 2024)（基于超过 15 万 token 的小说的 580 个问题），取问答 F1 与选择题准确率的平均。该基准上没有调过任何设置。

密切块打包的优势与 LongBench 上方向一致（合并后 +1.21，各组合在 +0.84 到 +1.56 之间），但在 580 个问题上不显著。只用 1,024 个 token，DedupPack 就达到或超过了读取该书 3 万 token 的模型（Qwen2.5-14B 上 38.03 对 36.44，Llama-3.1-8B 上 38.36 对 38.16）。

6.6 与完整上下文相差多少？

在 2,048 token 时，DedupPack 在三个模型上都与读取完整上下文在统计上没有差别：配对差值在 Qwen2.5-7B 上为 +0.56 [−1.46, 2.56]，在 Llama-3.1-8B 上为 −0.35 [−2.39, 1.65]，在 Qwen2.5-14B 上为 −0.10 [−2.04, 1.82]。打包上下文约为平均提示长度的六分之一（2,048 对 1.25 万 token）。在三个多跳任务上，它显著优于完整上下文：包括 Qwen2.5-32B 在内的四个模型合并后高 3.68 F1 [1.50, 5.91]（$p = 0.001$），并且在每个模型上都更高（MH F1 分别为 49.2 对 44.4、45.8 对 43.7、56.0 对 52.6、57.5 对 53.2；其中三个模型单独显著），这与“长输入会稀释真正重要的证据”的观察一致 (Liu et al., 2024a)。在 1,024 token 时，DedupPack 达到完整上下文 F1 的 93–95%。

6.7 更大的模型能弥补吗？

更强的阅读模型也许不那么容易被重复文字干扰，那样打包浪费就只是小模型的问题。但模型再强，也读不到根本没有被放进上下文的证据。因此我们用 Qwen2.5-32B-Instruct 重复了主要比较（由于单张 40 GB GPU 放不下 bf16 权重，使用官方发布的 AWQ 4-bit 版本；两种打包器使用同一个模型）。在 512、1,024 和 2,048 token 时，DedupPack 分别比 BGE 切块打包高 +3.36 [1.58, 5.18]、+2.67 [0.92, 4.48] 和 +3.07 [1.40, 4.76] F1（均 $p < 0.005$；表 7）。在 Qwen2.5 的 7B、14B 和 32B 上，提升没有随模型变大而缩小，在 1,024 和 2,048 token 时还是在最大的模型上最大。在 2,048 token 时，32B 模型配合 DedupPack 得到 49.15 F1，而 3 万 token 的完整上下文为 47.79（+1.36 [−0.61, 3.39]，不显著），即达到完整上下文 F1 的 103%。在同一张 GPU 上生成 1,150 个答案，用 2,048 token 的 DedupPack 上下文约需 20 分钟，用完整上下文约需 130 分钟，实测端到端加速 6.5 倍。

相对 BGE 切块的提升	512	1,024	2,048
Qwen2.5-7B	+3.69	+1.79	+2.62
Qwen2.5-14B	+4.17	+1.98	+2.21
Qwen2.5-32B（AWQ）	+3.36	+2.67	+3.07
32B：DedupPack / 切块	40.88 / 37.52	45.07 / 42.40	49.15 / 46.08

表 7: DedupPack（冻结的混合配置）相对 BGE 切块打包的提升，没有随阅读模型从 7B 增大到 32B 而缩小。最后一行：Qwen2.5-32B 的宏平均 F1；其完整上下文参考为 47.79。

打包器（BGE 排序，1,024 token）	Qwen-14B	Llama-8B	本步 Δ	重复率
切块打包	42.34	39.03		22.8
+ 合并重叠区间	44.42	41.17	+2.11***	1.2
+ 按原文顺序输出	44.75	41.05	+0.11	1.2
+ 句子对齐补全（= DedupPack）	45.18	41.67	+0.52	1.3
DedupPack 去掉回填	44.84	41.24	−0.38	1.3
DedupPack 去掉合并	43.32	40.10	−1.71***	19.7
对照（与切块打包比较）				
MMR 重排序 + 切块打包	43.21	39.95	+0.90	–
跳过重叠块（不合并）	44.24	40.26	+1.57	–

表 8: 固定 BGE 排序的逐步叠加消融。本步 Δ：与上一行的配对 F1 差值（下方两行：与 DedupPack 比较），两个模型合并（***：$p < 0.001$；每个模型单独也显著）。重复率：打包上下文中重复部分所占比例（%，Qwen 分词器）。对照：与切块打包的差值，两个模型合并。

6.8 证据级压缩与 token 级压缩

在相同的 1,024 token 精确预算下，LLMLingua-2 只有 24.6–28.4 F1，三个模型合并后 DedupPack 比它高 15.27 F1 [13.90, 16.64]。在大约 12 倍的压缩率下，token 级压缩器必须删去每个句子的大部分内容，包括抽取式问答需要照抄的人名、数字和虚词；而 DedupPack 把预算用在少数几段完整的文字上。附录 E 显示 Selective Context 和 LongLLMLingua 也呈现同样的排序。

6.9 去重是真正起作用的成分

为了找出是哪个操作带来了提升，我们从标准切块打包出发，每次加入一个操作，同时固定 BGE 排序，使变化的只有打包方式（表 8；我们重新实现的切块打包与表 4 中的基线相差不超过 0.04 F1）。仅合并重叠区间一步就把 F1 提高 2.11 [1.16, 3.09]（$p < 0.001$），在每个模型上单独也显著，并且几乎消除了全部重复（22.8%→1.2%）。它占全部 2.74 F1 提升的 77%。按原文顺序输出和句子对齐补全分别再

增加 0.11 和 0.52 F1，回填增加 0.38；这些步骤单独都不显著。反过来，从完整打包器中去掉合并会损失 1.71 F1 [0.76, 2.66]（$p < 0.001$），重复率回升到 20%。仅合并一步在其他预算下也显著（512 token 时 +1.47，2,048 token 时 +1.64，均 $p \leq 0.002$）。我们保留其余几个次要操作，因为它们从未造成显著损害，并且让打包上下文更易读；但证据清楚地表明，准确率的提升来自不为同样的文字付费两次。

是去重，还是连续性？ 合并同时做了两件事：去掉重复的文字，以及把碎片拼成更长的连续段落。为了区分两者，我们加了一个只去重的对照：按排名顺序打包文本块，但只要某个块补全后的范围与已选的块重叠，就跳过它，因此片段仍然短小而分散。它比切块打包高 1.57 F1（Qwen2.5-14B 上 +1.90，Llama-3.1-8B 上 +1.23），约为合并带来提升（+2.11）的四分之三；把片段拼接起来贡献了其余部分。通用的去冗余选择方法 MMR (Carbonell and Goldstein, 1998)（TF-IDF 冗余度，$\lambda = 0.7$）只提升 0.90，约为完整 DedupPack 提升的三分之一：惩罚相似的块，并不等于从不为同样的 token 付费两次。MMR < 只去重 < 合并这一顺序在两个模型上一致，不过三个对照之间的差异单独都不显著。

重复越多，提升越大。 如果提升来自去重，那么切块打包上下文冗余越多的样本应该获益越多。我们按切块打包上下文的重复率把测试样本分成三等份。在 512 token 时，DedupPack 相对切块打包（相同 BGE 排序）的 F1 提升从冗余最少的三分之一中的 −0.2 和 +1.7，增长到冗余最多的三分之一中的 +6.8 和 +7.1（分别对应 Qwen2.5-7B 和 Llama-3.1-8B）；在 512 和 1,024 token 时，两个模型上答案覆盖率的提升都随重复率单调增加（例如 Qwen2.5-7B 在 512 token 时依次为 +1.0、+3.1 和 +7.8 个百分点）。同样的机制也解释了 InfiniteBench 上提升较小的原因（第 6.5 节）：那里相关段落分散在整部小说中，切块打包只有 8–11% 的上下文是重复的。

6.10 成本

表 9 在受控条件下测量了各上下文构建方法的成本。DedupPack 在共享的 BGE 编码（GPU 上 0.15 秒）之外，只增加 0.40 秒单核 CPU 时间，比切块打包多 0.13 秒，并且除了 3,300 万参数的编码器之外，既不需要训练，也不需要额外显存。Provence 和 LLMLingua-2 需要一个训练过的剪枝或压缩模型，端到端耗时约为 1.6 倍；LongLLMLingua 用一个 80 亿参数的语言模型给长上下文打分，每个样本需要 21 秒和 18 GB 显存——成本约为 DedupPack 的 40 倍，F1 却低 2.5。每个 DedupPack 上下文都满足精确预算（最多 1,024 token），而且几乎把预算填满（平均 1,014 token，切块打包为 987）。

方法	需训练	额外模型	秒/样本	设备	显存
BGE-small 编码（共享）	否	33M	0.15	GPU	0.3 GB
+ 切块打包	否	–	+0.27	CPU	–
+ DedupPack	否	–	+0.40	CPU	–
+ 重排序器（前 100）	否	568M	+0.16	GPU	1.3 GB
+ Provence 剪枝	是	DeBERTa-v3	+0.76	GPU	3.4 GB
LLMLingua-2	是	560M	0.87	GPU	3.3 GB
LongLLMLingua	否	8B	21.05	GPU	18.4 GB

表 9: 构建 1,024 token 上下文的成本（随机抽取 100 个 LongBench 测试样本，逐个处理，独占一块 A100；DedupPack 和切块打包在单个 CPU 核上运行）。由于预算相同，所有方法在下游模型上的预填充成本相同，约为完整上下文的十二分之一。

7 讨论

为什么打包如此重要？ 在精确预算下，每个重复的 token 都是一个没有用在证据上的 token。标准打包对每个高排名块独立地进行补全，但高排名块之间并不独立：它们聚集在相关段落中，所以补全互相重叠，四分之一的预算被花了两次。合并补全区间消除了这一浪费，单这一步就值约 2 F1，与把 BGE-small 换成 BGE-large 或加入交叉编码重排序器的收益相当。从构造上就避免重叠的简单设计（例如互不重叠的句子块）能拿回部分提升，这与我们的诊断一致；而 DedupPack 可以接在任何现有的切块方式和排序器之后实现这一点，无需重建索引。

对压缩研究的启示。 大多数上下文压缩工作在一个固定、简单的打包器上比较排序器或学习型打分器。我们的结果表明，这类比较不仅应固定打包器，还应使用强的打包器，因为更好的打包器会给每种排序器带来相近的提升。DedupPack 只需要下游分词器，不增加任何模型调用，因此可以作为任何检索器或学习型打分器之后的默认打包器。

部署。 DedupPack 无需训练，在调用模型前运行，输出按原文顺序排列的逐字原文，因此压缩后的上下文可以展示给用户，并能追溯到原始出处。

实践意义。 DedupPack 可以直接替换打包这一步：不需要训练，除排序器外不需要任何模型，每个请求只增加约 0.4 秒 CPU 时间（第 6.10 节）。它节省下来的预算可以直接换算成输入 token。在一半的预算下，DedupPack 就几乎追平了切块打包：三个模型合并，1,024

token 的 DedupPack 得 41.98 F1，而 2,048 token 的切块打包得 42.37；在 Llama-3.1-8B 上 DedupPack 甚至更高（40.98 对 40.85）。在预算之间插值可知，切块打包需要大约 1.6–2 倍的 token，才能达到 1,024 token DedupPack 的精度。与完整上下文相比，2,048 token 的 DedupPack 上下文只用六分之一的输入就保留了 99–101% 的 F1，预填充开销和每个请求的 KV 缓存也大致按同样比例减少；在 Qwen2.5-32B 上，回答整个测试集所需的 GPU 时间比完整上下文少 6.5 倍。收益最大的是对检索命中做邻近扩展的流水线，例如句子窗口检索或父文档检索，我们在这类设置中测得 9–30% 的重复；而在来自许多不同文档、互不重叠的普通切块中，重复很少，可去除的也就不多。

8 结论

我们研究了在以下游模型自身 token 计量的严格预算下的长上下文问答，并表明“把排序后的片段变成上下文”这一步是一个重要但研究不足的杠杆。标准打包把 15–30% 的预算花在重复文字上；DedupPack 的核心是合并重叠证据、使任何文字都不被付费两次，它在我们测试的全部 9 个模型–预算组合上都优于稠密切块打包，三个模型合并后领先 1.9–3.7 F1；它能迁移到词法、稠密和混合排序器上，并在 2,048 token 时在三个 LLM 上与完整上下文持平。我们希望这能促使研究者把打包当作上下文压缩中的一等组件来对待。

局限性

DedupPack 在预填充之前缩短输入，并不是 KV 缓存方法。我们的实验涵盖 LongBench 上的英文抽取式问答，使用三个 7–14B 参数的指令模型和 512–2,048 token 的预算；其他语言、摘要等生成任务以及大得多的模型上的表现可能不同。开发集只包含两个多跳任务，方法在测试前就已冻结；更广泛的开发集可能会导致选择不同的排序器，不过我们的结果表明排序器的影响小于打包器。除合并之外，其余各打包操作单独的影响都很小，且不显著。稠密和混合排序器在推理时需要一个小型编码器。

伦理声明

上下文压缩可能删去用户认为重要的信息，从而导致回答不完整或错误；在高风险场景中，压缩后的上下文应展示给用户或与原文核对。DedupPack 对所有纳入的文字逐字保留，使这类核对很容易进行。所有数据集都是公开的研究基准，按其许可使用。

References

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2023. Longbench: A bilingual, multitask benchmark for long context understanding. arXiv preprint arXiv:2308.14508.

Jaime Carbonell and Jade Goldstein. 1998. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, pages 335–336.

Nadezhda Chirkova, Thibault Formal, Vassilina Nikoulina, and Stéphane Clinchant. 2025. Provence: Efficient and robust context pruning for retrieval-augmented generation. In The Thirteenth International Conference on Learning Representations.

Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. 2021. A dataset of information-seeking questions and answers anchored in research papers. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pages 4599–4610.

Bradley Efron and Robert J. Tibshirani. 1993. An Introduction to the Bootstrap. Chapman and Hall/CRC.

Matoš Eibich, Shivay Nagpal, and Alexander Fred-Ojala. 2024. ARAGOG: Advanced RAG output grading. arXiv:2404.01037.

Aaron Grattafiori et al. 2024. The Llama 3 herd of models. arXiv preprint arXiv:2407.21783.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In Proceedings of the 28th International Conference on Computational Linguistics, pages 6609–6625.

Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics, pages 874–880.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. Llmlingua: Compressing prompts for accelerated inference of large language models. arXiv preprint arXiv:2310.05736.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang,

and Lili Qiu. 2024. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. arXiv preprint arXiv:2310.06839.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, pages 6769–6781.

Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. 2018. The NarrativeQA reading comprehension challenge. Transactions of the Association for Computational Linguistics, 6:317–328.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In Advances in Neural Information Processing Systems, volume 33, pages 9459–9474.

Yucheng Li, Bo Dong, Chenghua Lin, and Frank Guerin. 2023. Selective context: Efficient context selection for large language models. arXiv preprint arXiv:2304.12102.

Yucheng Li, Ying Huang, Bowen Yang, Bharat Venkitesh, Andrea Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. Snapkv: Llm knows what you are looking for before generation. arXiv preprint arXiv:2404.14469.

Barys Liskavets, Maxim Ushakov, Shuvendu Roy, Mark Klibanov, Ali Etemad, and Shane K. Luke. 2025. Prompt compression with context-aware sentence encoding for fast and improved LLM inference. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 39, pages 24595–24604.

Jerry Liu. 2022. LlamaIndex. https://github.com/run-llama/llama_index.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024a. Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics, 12:157–173.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024b. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. arXiv preprint arXiv:2402.02750.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In Findings of the Association for Computational Linguistics: ACL 2024, pages 963–981.

Qwen Team. 2024. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: Bm25 and beyond. Foundations and Trends in Information Retrieval, 3(4):333–389.

Sietse Schelpe. 2026. Byte-exact deduplication in retrieval-augmented generation: A three-regime empirical analysis across public benchmarks. arXiv:2605.09611.

Nataraj Agaram Sundar and Tejas Morabia. 2026. Self-conditioned positional HNSW for overlap-aware retrieval in chunked-document RAG systems: Method and industrial evidence-quality audit. arXiv:2606.01542.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop questions via single-hop question composition. Transactions of the Association for Computational Linguistics, 10:539–554.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, pages 10014–10037.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023a. Efficient streaming language models with attention sinks. arXiv preprint arXiv:2309.17453.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023b. C-pack: Packaged resources to advance general Chinese embedding. arXiv preprint arXiv:2309.07597.

Wenhan Xiong, Xiang Lorraine Li, Srinivasan Iyer, Jingfei Du, Patrick Lewis, William Yang Wang, Yashar Mehdad, Wen-tau Yih, Sebastian Riedel, Douwe Kiela, and Mohit Bansal. 2021. Answering complex open-domain questions with multi-hop dense retrieval. In Proceedings of the International Conference on Learning Representations.

Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. Recomp: Improving retrieval-augmented lms with compression and selective augmentation. In Proceedings of the International Conference on Learning Representations.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pages 2369–2380.

Chanwoong Yoon, Taewhoo Lee, Hyeon Hwang, Minbyul Jeong, and Jaewoo Kang. 2024. CompAct: Compressing retrieved documents actively for question answering. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 21424–21439.

Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. 2024. ∞Bench: Extending long context evaluation beyond 100K tokens. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers).

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Re, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. arXiv preprint arXiv:2306.14048.

A 实现细节

算法 1 DedupPack

Require: 问题 q、上下文 C、预算 B、分词器 τ、排序器 f
把 C 切成 256 token 的页和 64 token 的片段（步长 32）
$P \leftarrow$ 按 $f(q, s)$ 排名前 60 的片段
$\mathcal{S} \leftarrow \emptyset$
for 按顺序取 $s \in P$ do
  $R \leftarrow \mathcal{S} \cup \{s\}$ 中各片段经句子补全后的区间（每侧 ≤32 token，不跨页）
  合并 R 中重叠的区间；在前缀之后按原文顺序渲染
  if $|\text{渲染结果}|_\tau \leq B$ then $\mathcal{S} \leftarrow \mathcal{S} \cup \{s\}$
  end if ▷ 否则跳过 s 继续（回填）
end for
return $\mathcal{S}$ 的渲染结果，必要时截断到 B 个 token

推理使用 vLLM 0.7.3，bfloat16 权重，运行在 NVIDIA A100-40GB GPU 上（14B 模型的完整上下文实验使用双卡张量并行）；压缩上下文的模型窗口为 4,096 token，完整上下文参考为 32,768 token。本文所有数字对应的代码、打包上下文和逐样本预测都将公开。

B 开发集结果

表 11 报告了开发集（去掉测试问题的 HotpotQA-E 和 2WikiMultihopQA-E，584 个样本，1,024 token，4-bit Qwen2.5-7B-Instruct）上的宏平均 F1。这些结果决定了冻结配置；混合排序器加句子补全是开发集上最好的配置，比 BGE 切块打包高 5.26 F1 [2.34, 8.23]。

项目	取值
页 / 片段	256 个正则 token / 64，步长 32
词法排序器	$0.5\,z$(BM25) + $0.5\,z$(TFIDF)；k_1=1.2，b=0.75；次线性 TF，平滑 IDF
稠密排序器	bge-small-en-v1.5，CLS 池化，ℓ_2 归一化，加查询指令
混合排序器	z(词法) + z(稠密)
优先列表	前 60 个片段
句末标点	. ？！
补全半径	每侧 32 个正则 token，不跨页
纳入方式	按 τ 计量渲染后的上下文，回填
输出方式	原文顺序，空行分隔

表 10: 实现常数（全部在测试前冻结）。

排序器（均含合并 + 回填）	固定 ±32	句子补全
词法	45.37	47.90
BGE	48.88	50.12
混合	48.17	51.19
BGE 切块	45.93	
配对打包	45.03	

表 11: 开发集宏平均 F1（1,024 token）。

C 各任务结果

表 12 列出了 1,024 token 下各任务的 F1。DedupPack（混合）在 512 / 2,048 token 时的多跳（MH）F1 分别为 39.47 / 49.20（Qwen2.5-7B）、37.02 / 45.78（Llama-3.1-8B）和 44.18 / 56.01（Qwen2.5-14B），BGE 切块对应为 34.84 / 46.37、33.52 / 44.14 和 40.02 / 52.57。

D 各模型配对检验

E 补充基线

在一次较早的运行中，使用相同的候选片段、提示模板、评分脚本和 1,024 token 精确预算，但使用 Qwen2.5-7B-Instruct 和 Meta-Llama-3-8B-Instruct 以及较早的软件环境，表 2 中的配对打包器比开头截断和中间截断分别高 13.1 和 13.6 F1，比 Selective Context 高 18.2，比 LLMLingua-2 高 11.3，比 LongLLMLingua 高 6.6，比 BM25 和 TF-IDF 切块打包分别高 2.1 和 1.9 F1（两模型平均，均 $p < 0.001$）。由于 DedupPack 在表 2 的每个组合上都优于配对打包器，这些基线同样被 DedupPack 超过；

模型	方法	Qasper	MF-en	HotpotQA	2Wiki	MuSiQue	NarrQA	MH	平均
Qwen2.5-7B	完整上下文	44.86	51.41	58.07	45.61	29.36	28.94	44.35	43.04
	LLMLingua-2	33.23	31.00	32.80	29.19	13.19	8.16	25.06	24.60
	配对打包	40.40	50.08	54.05	46.25	26.83	17.05	42.38	39.11
	BGE 切块	38.90	47.90	51.61	45.88	30.72	18.08	42.74	38.85
	DedupPack（BGE）	42.03	46.40	57.00	52.41	29.89	16.82	46.43	40.76
	DedupPack（混合）	38.73	49.31	57.10	48.32	33.39	16.97	46.27	40.64
Llama-3.1-8B	完整上下文	45.28	54.80	55.79	44.14	31.26	27.52	43.73	43.13
	LLMLingua-2	34.31	33.35	37.35	31.53	20.40	13.61	29.76	28.43
	配对打包	38.33	50.09	49.47	45.11	24.25	19.46	39.61	37.79
	BGE 切块	40.65	48.49	47.33	48.11	28.50	20.86	41.31	38.99
	DedupPack（BGE）	41.79	49.78	56.18	49.58	31.84	20.85	45.87	41.67
	DedupPack（混合）	41.28	52.99	51.86	49.42	30.37	19.95	43.89	40.98
Qwen2.5-14B	完整上下文	45.46	53.86	62.00	58.52	37.21	28.50	52.58	47.59
	LLMLingua-2	30.93	32.71	38.34	34.15	18.34	8.12	30.27	27.10
	配对打包	39.37	49.88	56.88	52.25	31.82	19.92	46.98	41.69
	BGE 切块	39.75	49.05	55.42	53.34	35.28	21.19	48.02	42.34
	DedupPack（BGE）	41.29	50.37	60.59	60.29	37.33	21.18	52.74	45.18
	DedupPack（混合）	40.70	53.03	58.78	55.52	36.65	21.24	50.32	44.32

表 12: 1,024 token 下各任务的 F1（完整上下文：最多 3 万 token）。

模型	比较	512	1,024	2,048
Qwen2.5-7B	DedupPack − BGE 切块	+3.69 [1.69, 5.68]	+1.79 [−0.08, 3.65]	+2.62 [0.82, 4.49]
	DedupPack − 配对打包	+2.86 [1.09, 4.59]	+1.53 [−0.14, 3.17]	+2.92 [1.26, 4.58]
	DedupPack-BGE − BGE 切块	+2.59 [0.91, 4.29]	+1.91 [0.21, 3.64]	+2.17 [0.43, 3.96]
Llama-3.1-8B	DedupPack − BGE 切块	+3.17 [1.14, 5.17]	+1.99 [0.20, 3.84]	+1.93 [0.08, 3.80]
	DedupPack − 配对打包	+2.39 [0.69, 4.07]	+3.19 [1.48, 4.89]	+3.18 [1.53, 4.80]
	DedupPack-BGE − BGE 切块	+2.89 [1.24, 4.56]	+2.68 [1.06, 4.38]	+2.73 [1.03, 4.44]
Qwen2.5-14B	DedupPack − BGE 切块	+4.17 [2.27, 6.12]	+1.98 [0.20, 3.85]	+2.21 [0.45, 3.99]
	DedupPack − 配对打包	+3.71 [2.15, 5.26]	+2.63 [1.17, 4.12]	+3.27 [1.65, 4.90]
	DedupPack-BGE − BGE 切块	+2.95 [1.38, 4.53]	+2.84 [1.25, 4.49]	+2.81 [1.12, 4.59]

表 13: 各模型的配对 bootstrap 差值（F1，95% 区间）。DedupPack 指冻结的混合配置。

由于它们是在不同环境中运行的，我们把它们放在这里而不是主表中。